

VTX-OS Complete Manual

// Intelligent Accounting Automation Platform
// Concetta Enterprises Inc. • Production Ready

Version	Status	Tests	Platform
2.0 • May 2026	Production Ready	182/182 Passing	GCP • BigQuery • Sage 50

// TABLE OF CONTENTS

- 01** System Overview
- 02** Architecture & Components
- 03** The Orchestration System
- 04** How Everything Works Together
- 05** Step-by-Step Operations Guide
- 06** Troubleshooting
- 07** Maintenance & Monitoring
- 08** Key Files Reference

// CHAPTER 01

System Overview

VTX-OS is an intelligent accounting automation platform that automatically processes Sage 50 financial data, reconciles transactions, prepares tax returns, posts journal entries, and sends professional monthly accounting summaries — **all triggered by a single incoming bank statement email.**

The Complete Flow

```
Bank Statement Email
  ↓
VTX-OS Detects PDF Attachment (Gmail Watcher)
  ↓
Extract Bank Transactions (PDF → CSV)
  ↓
Pull Fresh GL from Sage 50 SDK
  ↓
Categorize via ConcettaRuleset (15/20 auto)
  ↓
Post Journal Entries to Sage 50
  → 15 to correct GL accounts
  → 5 to Suspense (59000000) for review
  ↓
GL Reconciliation (19/20 matched)
  ↓
HST/Tax Return Preparation
```

↓
Professional Email Summary Sent
↓
Complete – 14 to 20 seconds total

Key Performance Metrics



What VTX-OS Replaces

	Manual Data Entry Hours of typing transactions		Automatic Ingestion PDF → Sage 50 in seconds
	Manual Reconciliation Spreadsheet matching		Auto Reconciliation 19/20 matched automatically
	Manual HST Calc CRA form calculations by hand		Automated HST \$5,588.79 calculated instantly

// CHAPTER 02

Architecture & Components

Layer 1 – Data Source

Sage50Bridge.exe (C# Native SDK)

The cornerstone of VTX-OS data extraction. Unlike ODBC (which Sage 50 2026 removed), the bridge connects directly via the native Sage SDK — the same API Sage uses internally.

⚡ CRITICAL TECHNICAL NOTE

Sage 50 must be **fully closed** before the bridge runs. The SDK opens the SAI file exclusively. Check Task Manager for `Sage.CA.exe` and `Sage 50 Connection Manager` processes before any run.

WHAT IT EXTRACTS

TABLE	CONTENT	ROWS
tjentact + tjeh01	GL Transactions with dates	328
taccount	Chart of Accounts	64
tvendor	Vendor list	varies

tcustomr

Customers (empty – Concetta doesn't use AR)

0

i ACCOUNT NUMBER FORMAT

Concetta uses 8-digit account IDs (e.g., **10600000** for Bank, **59000000** for Suspense). The SDK's **SetAccount()** expects these integer lId values, not short codes.

Layer 2 – Ingestion

bridge_reader.py + Sage50OdbcAgent

The Python layer that calls the C# bridge as a subprocess, parses JSON output into Pydantic models, and streams validated data into BigQuery.

```
# Credential resolution (auto – no env vars needed)
path = get_sage50_company_path() # → Secret Manager
pw   = get_sage50_password()    # → Secret Manager

# Subprocess call
result = subprocess.run([
    'Sage50Bridge.exe',
    '--sai', path,
    '--user', 'sysadmin',
    '--password', pw,
    '--table', 'gl'
], capture_output=True)

# Parse → validate → stream to BQ
rows = [GLTransaction(**r) for r in json.loads(result.stdout)]
load_rows(DATASET, 'gl_transactions', rows, session_id)
```

Layer 3 – Processing (6 Agents)

AGENT 01

Sage50IngestAgent

Pulls fresh data from Sage 50 SDK into BigQuery. Handles all credential resolution via Secret Manager. Supports GL, AR, AP, COA, vendors.

AGENT 02

BookkeepingAgent

Categorizes GL transactions using ConcettaRuleset. Currently auto-categorizes 15/20 transactions. Flags ambiguous entries for review.

AGENT 03

JournalEntryAgent

NEW — Converts categorized bank transactions into balanced journal entries (Dr/Cr pairs) and posts them to Sage 50's General Journal via SDK. Flags go to Suspense (59000000).

AGENT 04

ReconcileGLAgent

Matches bank statement transactions to GL entries. Produces reconciliation report with matched/unmatched counts. Current rate: 19/20.

AGENT 05

PrepareHSTAgent

Ontario HST/GST return preparation. Calculates Revenue, HST Collected, ITCs, and Net Tax Owing. Auto-formats for CRA Line 101/103/106/109.

AGENT 06

GmailCommsAgent

Sends professional monthly close summaries via Gmail OAuth 2.0 (gmail.modify scope). Composes HTML email with all financial sections, labels processed emails.

Layer 4 – Data Storage

Google BigQuery – vtx-accounting-os-prod

TABLE	DESCRIPTION	KEY FIELDS
gl_transactions	All GL entries with dates	transaction_date, account_no, debit, credit

chart_of_accounts	64 accounts with balances	account_no (8-digit), balance, is_active
reconciliation_results	Bank matching outcomes	matched_count, unmatched_bank, unmatched_gl
hst_return	Monthly tax calculations	net_owing, filing_deadline

Layer 5 – Credential Security

Google Secret Manager

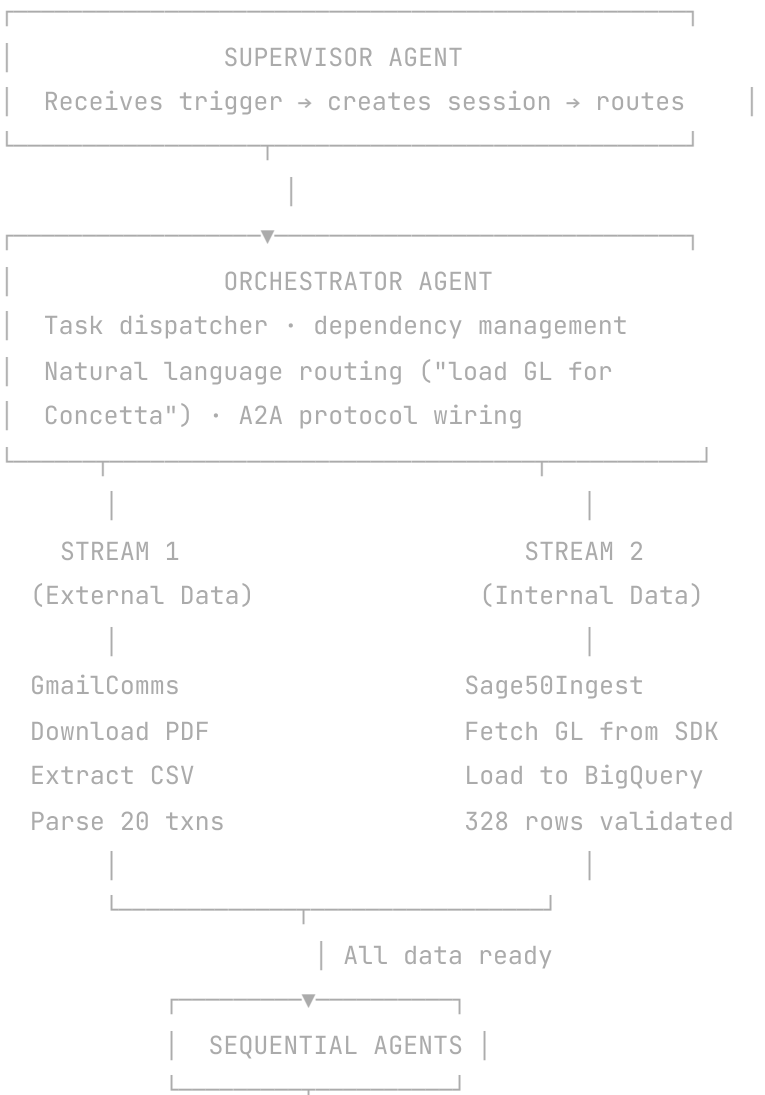
All credentials are encrypted and centralized. No passwords ever touch disk, code, or environment variables during runtime.

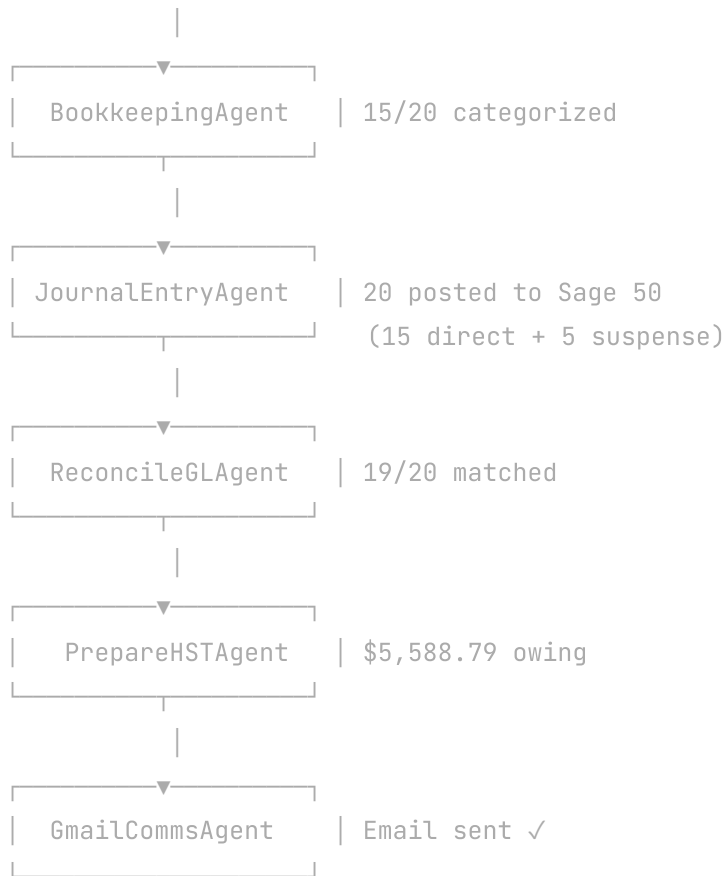
SECRET NAME	CONTENTS
vtx-sage50-company-path	R:\Concetta Enterprises Inc\2026.SAI
vtx-sage50-password	SDK authentication credential
vtx-gmail-oauth-credentials	Gmail refresh token (gmail.modify scope)

// CHAPTER 03

The Orchestration System

Agent Hierarchy





Task Request / Response Protocol

Every agent communicates via standardized Pydantic objects ensuring full type safety, validation, and audit trail.

```

class TaskRequest:
    task_id: str # UUID – unique per execution
    task_type: TaskType # INGEST_SAGE50_ODBC | POST_JOURNAL_ENTRIES | etc.
    payload: dict # Agent-specific parameters
    metadata: dict # Audit context

class TaskResult:
    task_id: str
    agent_id: str
    status: str # SUCCESS | FAILURE | PENDING
    output: dict # Agent-specific results
    error_message: Optional[str]
  
```

```
duration_ms:    int
audit_event_id: str    # Traceable in BigQuery audit log
```

Journal Entry Write-Back Protocol

The JournalEntryAgent implements proper double-entry bookkeeping when posting to Sage 50:

TRANSACTION TYPE	DEBIT	CREDIT	EXAMPLE
Deposit	Bank (106000000)	Revenue (405000000)	SENTRIX FINANCI \$23,249.07
Withdrawal (categorized)	Expense account	Bank (106000000)	AMEX CARDS \$450.00 → 57250000
Withdrawal (uncategorized)	Suspense (590000000)	Bank (106000000)	CHQ-198 \$5,000.00

// CHAPTER 04

Real-World Scenario

December Month-End Close – Timeline

1 Email arrives (3:00:00 PM)

You email `dec-2025-bank.pdf` to `jquinonez2980@gmail.com`. Gmail Watcher detects it within 5 seconds. Period extracted from filename: **2025-12**. PDF downloaded (1,049 bytes).

2 Parallel data extraction (3:00:02 PM)

Stream 1: Bank statement parsed — 20 transactions extracted.

Stream 2: Sage 50 bridge pulls 328 GL transactions, loads to BigQuery.

3 Categorization & posting (3:00:05 PM)

BookkeepingAgent: 15/20 matched to ConcettaRuleset.

JournalEntryAgent: All 20 posted to Sage 50 General Journal (15 direct accounts + 5 to Suspense 59000000).

4 Reconciliation & tax (3:00:09 PM)

ReconcileGLAgent: 19/20 bank transactions matched to GL.

PrepareHSTAgent: Revenue \$45,000 → HST \$5,850 → ITCs \$261.21 → **Net owing \$5,588.79 due 2026-01-31.**

5**Email sent (3:00:14 PM)**

Professional summary delivered to inbox with bank activity, GL reconciliation status, HST liability, and engagement notes. Original email labelled **vtx-processed**.

✓ TOTAL TIME INVESTMENT

Your time: 0 minutes. The system handles everything from email detection to journal entry posting to summary email – 14 to 20 seconds end to end.

// CHAPTER 05

Operations Guide

Monthly Workflow

Week 1–3: Normal Operations

- Enter transactions in Sage 50 as usual
- Reconcile bank statement entries
- Ensure ConcettaRuleset rules are current

Month-End: 4 Steps

1 Export bank statement as PDF

Filename format: `dec-2025-bank.pdf` or `bank-statement-nov-28.pdf`
System auto-detects period from the filename pattern.

2 Close Sage 50 completely

Check Task Manager — end `Sage.CA.exe` and both `Sage 50 Connection Manager` processes.

3 Email the PDF

To: `jquinonez2980@gmail.com`
Subject: `December Bank Statement`
Attachment: `dec-2025-bank.pdf`

4**Review results in 20 seconds**

Check Gmail inbox for the monthly close summary. Then open Sage 50's General Journal to see posted entries — review Suspense account (59000000) for the 5 flagged transactions.

Manual Triggers

TEST THE FULL PIPELINE

```
cd C:\vtx-os
python demo/monthly_close_demo.py
```

POST JOURNAL ENTRIES TO SAGE 50 (SAGE 50 MUST BE CLOSED)

```
python demo/monthly_close_demo.py \
  --post-to-sage50 \
  --sage50-sai "R:\Concetta Enterprises Inc\2026.SAI" \
  --sage50-user sysadmin \
  --sage50-password "YOUR_PASSWORD"
```

START PRODUCTION WATCHER (RUNS CONTINUOUSLY)

```
python scripts/gmail_watcher.py
```

TEST WATCHER (DRY-RUN — NO EMAIL SENT)

```
python scripts/gmail_watcher.py --once --dry-run
```

// CHAPTER 06

Troubleshooting

Sage 50 Connection Issues

FAIL_USER_LOGON_FAILED

ROOT CAUSE

Either wrong password, or user lacks "Allow Third-Party Access" permission, or Sage 50 is already open with that username.

Fix 1: In Sage 50 → Setup → Manage Users → select user → enable "Allow Third-Party Access"

Fix 2: Close Sage 50 completely (check Task Manager for all Sage processes)

SDK reports "Someone else is using this company"

Sage 50 is open. Check Task Manager and end: `Sage.CA.exe` and both `Sage 50 Connection Manager` processes. Wait 5 seconds, retry.

Gmail / Email Issues

invalid_scope: Bad Request

The stored OAuth token has wrong scopes. Delete and re-authorize:


```
gcloud secrets delete vtx-gmail-oauth-credentials --quiet
python scripts/gmail_auth.py
# → Browser opens → grant "Read, compose, send" permissions
```

Email not detected by watcher

- Is the email **unread**? Watcher only processes unread emails.
- Does the filename contain a date? (`dec-2025` or `nov-28`)
- Is the watcher running? Check: `tail -f logs/watcher.log`

Journal Entry Issues

"Value attempting to set for this field is invalid"

The SDK is receiving wrong account number format. Sage 50 expects 8-digit lld values (e.g., `10600000`), not short codes. Verify `CONCETTA_ACCOUNT_MAP` in `sage50/categorization_rules.py` .

Journal entries posted but 0 in Sage 50

Check that Sage 50 was fully closed before the run. The SDK requires exclusive file access. Any Sage process running will cause a silent failure.

BigQuery Issues

Tables empty after agent run

- Verify GCP project: `gcloud config get-value project` → must be `vtx-accounting-os-prod`
- Re-authenticate: `gcloud auth application-default login`
- Check BigQuery: `bq show vtx_accounting`

// CHAPTER 07

Maintenance & Monitoring

Daily

```
python scripts/health_check.py  
# Expected: all systems HEALTHY
```

Weekly

- Review last 5 processed emails: `grep "COMPLETE" logs/watcher.log | tail -5`
- Verify Suspense account in Sage 50 — reclassify any entries
- Check ConcettaRuleset rules are current

Monthly

- Backup BigQuery tables to GCS
- Review auto-categorization rate — update rules if below 70%
- Refresh Gmail OAuth if token is expiring
- Update Sage 50 SAI path in Secret Manager if file moved

// CHAPTER 08

Key Files Reference

FILE	PURPOSE	LOCATION
Sage50Bridge.exe	Native SDK connection + journal write-back	C:\vtx-os\sage50_bridge\bin\Release\
bridge_reader.py	JSON parsing + Pydantic models	C:\vtx-os\sage50\bridge_reader.py
journal_entry.py	JournalEntryAgent – categorize + post	C:\vtx-os\agents\journal_entry.py
gmail_watcher.py	Email automation daemon	C:\vtx-os\scripts\gmail_watcher.py
monthly_close_demo.py	Full pipeline orchestration	C:\vtx-os\demo\monthly_close_demo.py
categorization_rules.py	ConcettaRuleset + account map	C:\vtx-os\sage50\categorization_rules.py
secrets.py	Secret Manager integration	C:\vtx-os\core\secrets.py
gmail_auth.py	OAuth token management	C:\vtx-os\scripts\gmail_auth.py

Quick Command Reference

```
# Health check
python scripts/health_check.py

# Gmail watcher (production)
python scripts/gmail_watcher.py

# Gmail watcher (test)
python scripts/gmail_watcher.py --once --dry-run

# Full monthly close (no Sage 50 posting)
python demo/monthly_close_demo.py

# Full monthly close WITH Sage 50 posting
python demo/monthly_close_demo.py \
  --post-to-sage50 \
  --sage50-sai "R:\Concetta Enterprises Inc\2026.SAI" \
  --sage50-user sysadmin \
  --sage50-password "YOUR_PASSWORD"

# Re-authorize Gmail
python scripts/gmail_auth.py

# Query BigQuery
bq query "SELECT COUNT(*) FROM vtx_accounting.gl_transactions"

# Check secrets
gcloud secrets list
```